LAB # 02

Submitted By: Muhammad Ahmad Malik

Student ID: 41171

Submitted To: Mam Zilla Huma

Subject Name: Advance Data Base Management (LAB)

Submission Date: 10/24/2026

Lab Questions

Q#1. What is a heap file? How are records stored in it?

A heap file is the most basic way a database stores data. Records are stored in no particular order. When a new record is inserted, the database simply places it in the next available free space on the disk. It is fast for inserting, but slow for searching because you have to scan the whole file.

Q#2. What is ctid? Why does the ctid of a row change after an UPDATE?

ctid stands for "Current Tuple ID". It is a hidden system column that shows the exact physical location of a row (which page and which row number it is on). It changes after an UPDATE because PostgreSQL uses MVCC (Multi-Version Concurrency Control). It does not overwrite the old row; instead, it writes a brand new copy of the row at the end of the file, and marks the old one as "dead". Because it is a new physical copy, it gets a new ctid.

Q#3. Explain the difference between a B-tree index and a hash index. When would you use each one?

- **B-tree:** Keeps data sorted in a tree structure. It is great for equality (=), range searches (<, >, BETWEEN), and sorting (ORDER BY). Use it for general purposes and ranges.
- **Hash:** Uses a math formula to map values to a location. It is only good for exact equality searches (=). It cannot do ranges. Use it only when you need to look up exact values very fast.

Q#4. Why may PostgreSQL ignore an index on a very small table? How can you check this with EXPLAIN?

If a table is very small (e.g., 1 or 2 pages), the whole table is already in memory. Reading the whole table quickly (Sequential Scan) is actually faster than the overhead of looking up the index, finding the pointer, and then fetching the data. You can check this by putting EXPLAIN in front of your query. If it says "Seq Scan" instead of "Index Scan", PostgreSQL ignored the index.

Q#5. Give two disadvantages of creating too many indexes on a table.

1. **Wasted Disk Space:** Every index is a separate file on your hard drive that takes up space.
2. **Slower Writes:** Every time you INSERT, UPDATE, or DELETE a row, PostgreSQL has to update all the indexes on that table. Too many indexes will make your write operations very slow.

Programming Exercises

Q#1. Execute the program and write the ctid and output.

Code:

```
CREATE TABLE book_heap (
    book_id  INT,
    title    VARCHAR(50),
    price    INT
);

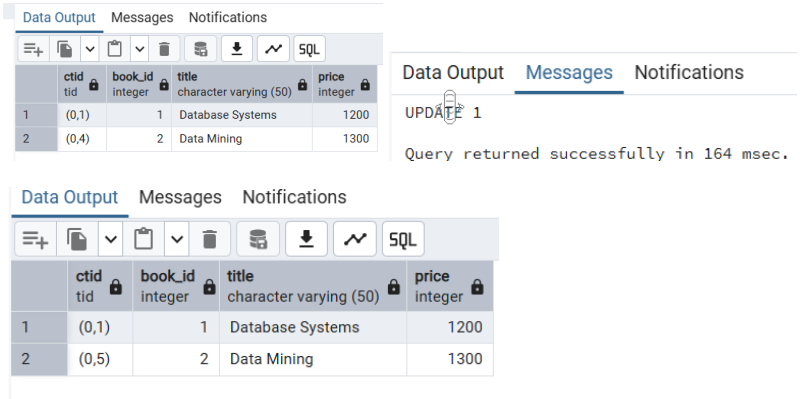
INSERT INTO book_heap VALUES
(1, 'Database Systems',  1200),
(2, 'Data Mining',       1500),
(3, 'Operating Systems', 1100);

SELECT ctid, * FROM book_heap;

UPDATE book_heap SET price = 1300 WHERE book_id = 2;
SELECT ctid, * FROM book_heap;

DELETE FROM book_heap WHERE book_id = 3;
SELECT ctid, * FROM book_heap;
```

OUTPUT:



Data Output | Messages | Notifications

	ctid tid	book_id integer	title character varying (50)	price integer
1	(0,1)	1	Database Systems	1200
2	(0,4)	2	Data Mining	1300

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 164 msec.
```

Data Output | Messages | Notifications

	ctid tid	book_id integer	title character varying (50)	price integer
1	(0,1)	1	Database Systems	1200
2	(0,5)	2	Data Mining	1300

- **Explanation:** This code creates a simple table and inserts 3 books, then performs an UPDATE on book 2 and a DELETE on book 3.

Q#2. Create product_heap and run VACUUM.

Code:

Query | Query History

```sql
CREATE TABLE product_heap (
    product_id INT,
    product_name VARCHAR(50),
    price INT,
    stock INT
);

INSERT INTO product_heap VALUES
(1, 'Laptop', 1500, 20),
(2, 'Mouse', 25, 100),
(3, 'Keyboard', 45, 50),
(4, 'Monitor', 200, 30),
(5, 'Printer', 120, 10);

SELECT ctid, * FROM product_heap;

UPDATE product_heap SET price = 180 WHERE product_id = 4;

DELETE FROM product_heap WHERE product_id = 2;
```

OUTPUT:

Data Output | Messages | Notifications

```
DELETE 1

Query returned successfully in 201 msec.
```

	ctid tid	product_id integer	product_name character varying (50)	price integer	stock integer
1	(0,1)	1	Laptop	1500	20
2	(0,3)	3	Keyboard	45	50
3	(0,5)	5	Printer	120	10
4	(0,6)	4	Monitor	180	30
5	(0,7)	1	Laptop	1500	20
6	(0,8)	2	Mouse	25	100
7	(0,9)	3	Keyboard	45	50
8	(0,10)	4	Monitor	200	30
9	(0,11)	5	Printer	120	10

Data Output | Messages | Notifications

```
UPDATE 2

Query returned successfully in 127 msec.
```

Explanation: This creates a heap table for products, inserts 5 rows, updates a price, and deletes a product.

Q#3. Create orders table and test indexes.

Code:

Query | Query History

```sql
CREATE TABLE orders (
    order_id INT,
    customer_id INT,
    amount INT
);

INSERT INTO orders
SELECT g, g % 1000, g % 5000
FROM generate_series(1, 100000) AS g;

CREATE INDEX idx_customer_btree ON orders(customer_id);

CREATE INDEX idx_order_hash ON orders USING hash(order_id);

SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';

EXPLAIN SELECT * FROM orders WHERE customer_id = 500;

EXPLAIN SELECT * FROM orders WHERE order_id = 500;
```

OUTPUT:

Data Output | Messages | Notifications

	QUERY PLAN text
1	Bitmap Heap Scan on orders (cost=15.88..574.37 rows=500 width=12)
2	Recheck Cond: (order_id = 500)
3	-> Bitmap Index Scan on idx_order_hash (cost=0.00..15.75 rows=500 widt...
4	Index Cond: (order_id = 500)

Data Output | Messages | Notifications

	QUERY PLAN text
1	Index Scan using idx_order_hash on orders (cost=0.00..8.02 rows=1 width...
2	Index Cond: (order_id = 500)

Explanation: This creates a table with 100,000 rows and builds a B-tree index on customer_id and a Hash index on order_id.

Q#4. Performance before/after index and hash range test.

Code:

```
Query  Query History
1   -- 1. Before Index
2   EXPLAIN ANALYZE SELECT * FROM orders WHERE amount = 2500;
3
4   -- 2. Create Index
5   CREATE INDEX idx_amount_btree ON orders(amount);
6   ANALYZE orders;
7
8   -- 3. After Index
9   EXPLAIN ANALYZE SELECT * FROM orders WHERE amount = 2500;
10
```

OUTPUT:

Data Output Messages Notifications

	QUERY PLAN text
1	Seq Scan on orders (cost=0.00..2041.00 rows=102 width=12) (actual time=0.025..12.680 rows=101.00 loop...
2	Filter: ((order_id >= 100) AND (order_id <= 200))
3	Rows Removed by Filter: 99899
4	Buffers: shared hit=541
5	Planning:
6	Buffers: shared hit=10
7	Planning Time: 0.114 ms
8	Execution Time: 12.701 ms

Showing rows: 1 t

	QUERY PLAN text
1	Bitmap Heap Scan on orders (cost=4.45..73.16 rows=20 width=12) (actual time=0.052..0.085 rows=20.00 loops=1)
2	Recheck Cond: (amount = 2500)
3	Heap Blocks: exact=20
4	Buffers: shared hit=22
5	-> Bitmap Index Scan on idx_amount_btree (cost=0.00..4.44 rows=20 width=0) (actual time=0.016..0.016 rows=20.00 loo...
6	Index Cond: (amount = 2500)
7	Index Searches: 1
8	Buffers: shared hit=2
9	Planning:
10	Buffers: shared hit=27
11	Planning Time: 3.924 ms
12	Execution Time: 0.116 ms

Total rows: 12 Query complete 00:00:00.154

Explanation: EXPLAIN ANALYZE is run for amount = 2500 before and after creating an index, showing a dramatic drop in execution time from a Seq Scan to an Index Scan.